

Laporan Analisis Fungsi CRUD

Aplikasi Cashier Latte

Pemetaan komprehensif implementasi operasi Create, Read, Update, dan Delete pada layer Cloud Firestore, SQLite Lokal, State Notifier, serta Antarmuka Pengguna (UI).

NAMA APLIKASI
Cashier BGA Co. Latte

FRAMEWORK & DATABASE
Flutter + Firestore & SQLite

STATUS IMPLEMENTASI
100% Aktif & Sinkron

1 Arsitektur Multi-Tier CRUD Aplikasi

Aplikasi Cashier Latte menerapkan arsitektur ganda database: **Cloud Firestore** sebagai sistem database utama yang sinkron secara real-time dan multi-device, serta **SQLite (sqflite)** sebagai modul database lokal terstruktur dan pembelajaran.

Alur Eksekusi Data CRUD (Optimistic UI & Real-Time Sync)



2 Matriks Entitas CRUD Utama

ENTITAS DATA	TIPE PENYIMPANAN	OPERASI CRUD YANG TERSEDIA	LOKASI FILE SOURCE
Menu Items (Katalog Produk)	Cloud Firestore	Create Menu, Read All / Filter, Update Menu, Soft/Hard Delete	database_helper.dart menu_data_store.dart
Categories (Kategori Menu)	Cloud Firestore	Add Category, Fetch Sorted, Rename/Update, Cascade Delete	database_helper.dart menu_data_store.dart
Transactions & Invoices	Cloud Firestore	Insert POS Order, Stream Invoices, Filter by Store, Fetch Detail	database_helper.dart gris_payment_screen.dart
Staff & Shift Roster	Cloud Firestore	Register Staff, Assign Shift, Update Status Hadir/Break, Delete	database_helper.dart staff_shift_screen.dart
User Login & Profile	Cloud Firestore + SQLite	Register, Query Login Auth, Edit Bio & Avatar, Remove User	database_helper.dart db_helper.dart
Store Branches (Cabang)	Cloud Firestore	Insert Store, Get All Stores, Delete Store by ID / Name	database_helper.dart store_showcase_screen.dart

3 Detail Implementasi CRUD: Menu Items & Kategori

Modul Menu dan Kategori merupakan inti dari aplikasi POS Cashier Latte. Terintegrasi penuh dengan **Firestore Collection** 'menu_items' dan 'categories' dengan sinkronisasi reaktif ke UI.

1. CREATE (Insert Menu & Category)

CREATE

- **Insert Menu:** Menambahkan menu baru lengkap dengan nama, harga, deskripsi, kategori, serta foto (Path Asset / Base64 Bytes).
- **Insert Category:** Membuat kategori baru dengan penomoranurut (sort order) otomatis.
- **Optimistic State:** Ditambahkan langsung ke ValueNotifier agar UI merespons Oms tanpa menunggu latency jaringan.

2. READ (Fetch & Real-time Stream)

READ

- **Get All Menu:** Memuat seluruh menu yang berstatus `is_active == 1`.
- **Get by Category:** Filter instan berdasarkan kategori tab yang dipilih pengguna (Food, Drink, Snack, Dessert).
- **Real-time Stream:** Listener Firestore aktif meng-update katalog jika ada perubahan dari admin lain.

3. UPDATE (Edit Menu & Rename Category)

UPDATE

- **Update Menu:** Memperbarui harga, deskripsi, foto baru, atau perpindahan kategori produk.
- **Rename Category:** Mengubah nama kategori dan secara otomatis meng-update seluruh menu yang berelasi dengan kategori tersebut (Cascade sync).

4. DELETE (Hapus Menu & Kategori)

DELETE

- **Delete Menu Item:** Menghapus dokumen menu berdasarkan ID spesifik dari Firestore.
- **Delete Category:** Menghapus kategori beserta membersihkan item terkait agar integritas data tetap konsisten.
- **UI Action:** Disediakan konfirmasi modal sebelum aksi penghapusan permanen.

SNIPPET CODE CRUD MENU (LIB/HALAMAN1/DATABASE/DATABASE_HELPER.DART)

DART / FIRESTORE

```
/// 1. CREATE: Tambah Item Menu Baru
Future<int> insertMenuItem(MenuItemModel item) async {
  final map = item.toMap();
  final newId = item.id ?? DateTime.now().millisecondsSinceEpoch;
  map['id'] = newId;
  if (item.imageBytes != null) map['image_bytes'] = _bytesToBase64(item.imageBytes);
  await _menuItemsCol.doc('menu_' + newId.toString()).set(map);
  return newId;
}

/// 2. READ: Ambil Semua Menu Aktif
Future<List<MenuItemModel>> getAllMenuItems() async {
  final snapshot = await _menuItemsCol.where('is_active', isEqualTo: 1).get();
  return snapshot.docs.map((doc) => MenuItemModel.fromMap(doc.data())).toList();
}

/// 3. UPDATE: Perbarui Data Menu
Future<bool> updateMenuItem(MenuItemModel item) async {
  final map = item.toMap();
  await _menuItemsCol.doc('menu_' + item.id.toString()).set(map, SetOptions(merge: true));
  return true;
}

/// 4. DELETE: Hapus Menu Berdasarkan ID
Future<bool> deleteMenuItem(int id) async {
  final snapshot = await _menuItemsCol.where('id', isEqualTo: id).get();
  for (final doc in snapshot.docs) { await doc.reference.delete(); }
  return true;
}
```

4 Detail Implementasi CRUD: Transaksi POS & Kasir

Transaksi POS (Orders & Invoices)

CREATE / READ

- **CREATE (Checkout POS):** Menyimpan transaksi ke Firestore dengan rincian item belanja (array objek), subtotal, pajak PPN 10%, metode pembayaran (QRIS/Cash), serta nomor Invoice unik otomatis (contoh: #INV-20260914-001).
- **READ (Riwayat & Struk):** Stream transaksi langsung ke halaman Riwayat, pencarian invoice per cabang toko (Bella Cafe, Central Perk, dll).

Staff & Jadwal Shift Karyawan

FULL CRUD

- **CREATE:** Form Tambah Karyawan baru (Nama, Role, No HP, Foto Avatar).
- **READ:** Roster harian shift (Pagi/Sore) berdasarkan tanggal aktif.
- **UPDATE:** Ubah status absensi (Hadir, Istirahat, Izin, Selesai Shift).
- **DELETE:** Hapus data staf atau jadwal shift lama.

5 Implementasi CRUD SQLite Lokal (Modul 8SQFLITE)

Modul `lib/8SQFLITE_CRUD/database/db_helper.dart` menggunakan engine SQL lokal murni dengan file database `pkpd.db` untuk mengelola autentikasi dan data pengguna secara offline.

OPERASI CRUD SQLITE (LIB/8SQFLITE_CRUD/DATABASE/DB_HELPER.DART)

SQLITE / SQFLITE

```
/// 1. CREATE: Insert data user ke tabel 'users'
Future<bool> registerUser(UserModelSQL pengguna) async {
  final db = await database;
  try {
    await db.insert('users', pengguna.toMap());
    return true;
  } catch (e) { return false; }
}

/// 2. READ: Query user untuk verifikasi login & get all users
Future<List<UserModelSQL>> getAllUsers() async {
  final db = await database;
  final List<Map<String, dynamic>> results = await db.query('users');
  return results.map((map) => UserModelSQL.fromMap(map)).toList();
}

/// 3. UPDATE: Update data user berdasarkan id
Future<bool> updateUser(UserModelSQL pengguna) async {
  final db = await database;
  int count = await db.update('users', pengguna.toMap(), where: 'id = ?', whereArgs: [pengguna.id]);
  return count > 0;
}

/// 4. DELETE: Hapus user berdasarkan id
Future<void> deleteUser(int id) async {
  final db = await database;
  await db.delete('users', where: 'id = ?', whereArgs: [id]);
}
```

6 Kesimpulan & Ringkasan Validasi

ASPEK PENGUJIAN	HASIL ANALISIS KODE	STATUS
Kesiapan Fungsi CRUD	Semua fungsi Create, Read, Update, Delete telah diimplementasikan lengkap di helper dan UI view.	Lengkap (100%)
Penanganan Error (Exception Safety)	Dilengkapi blok try-catch, fallback data default, serta logging debugPrint.	Aman & Terproteksi
Sinkronisasi Realtime	Mendukung stream snapshots Firestore dan ValueNotifier untuk pengalaman kasir responsif.	Real-Time Aktif

